Phase 1 - Project Report

CITS5501/3501 | Software Testing and Quality Assurance | Group 11

Team Members:

- Anto Melvin Mathew (Student No: 25057384) - Github: ANTOMELV
- Dip Gupta (Student No: 24735786) - Github: diptadg
- Sarwesh Kattel (Student No: 24769645) - Github: Sarwesh13
- Leon Lin (Student No: 23744478) - Github: enqinghayashi
- Chris Sajimon Joseph (Student No: 24692035) - Github: ChrisSajimon

Repository: GitHub repository

Planning and Organization

1. *Collaboration & Workflow Plan*

Our Collaboration & Workflow Plan is maintained in our GitHub repository. We use Microsoft Teams as our primary communication channel for day-to-day discussions and quick updates. The group meets during our scheduled Wednesday lab session (2–4pm) and will hold additional online meetings on Teams as needed. Tasks are divided among members and tracked through GitHub Issues. If a team member is unavailable due to illness or other commitments, their assigned tasks will be redistributed among the remaining members to keep the project on track.

2. *Task Tracking*

We use GitHub Issues to assign and track tasks. We chose it because it integrates directly with our repository and provides a simple way to assign responsibilities and monitor progress. All group members have previous experience using GitHub.

Language and Tooling

3. *Language Choice*

We have chosen Python. Everyone in the group is familiar with Python, and overall the team has more experience with Python than Java.

Our top three factors were:

1. Familiarity with the language
2. Team members' experience
3. Library support

Python should allow us to develop and debug the application quickly and gives us access to a large range of libraries. A possible disadvantage is that Python has lower performance than Java in some situations, and its dynamic typing can make some errors harder to catch early.

If we had to use Java, the main difficulty would be getting everyone equally comfortable with the language and its more verbose development process. Java's static typing and compile-time checking could, however, make some errors easier to identify.

4. *Build/Dependency Management Tool*

We plan to use uv for Python dependency and environment management. This will be the group's first time setting up a project with uv. We chose it because we found out it is lightweight, fast, and provides a straightforward way to manage dependencies and virtual environments.

Technical Risks

5. *Parts Most Likely to Fail*

The most likely thing to break is making requests to the website. The server might be slow, go down, or return an error instead of the page we expect. Parsing the HTML could also fail if the page layout is messy or different from what we expected. Downloading files is another risk because a link might look like it points to a PDF but actually gives us an error page or a broken file. We also need to watch out for circular links where page A links to page B and page B links back to page A, which could trap the scraper in a loop.

6. *External Factors*

The website could change its layout or move pages around after we finish building the scraper, which would break our code. They might also add login requirements, CAPTCHAs, or start blocking our requests if we send too many too fast. Documents could be deleted or renamed. On our end, updating a library we depend on could also cause things to stop working if the new version behaves differently.

7. *Easiest vs Hardest to Test*

The easiest parts to test would be things like searching through downloaded documents and checking file types, because we can create our own test data and know exactly what the output should be. The hardest parts to test would be the actual scraping, because it depends on a real website that we do not control. Testing things like retry logic and rate limiting is also difficult because they depend on timing and network conditions that are hard to recreate consistently.

8. *Assumptions About the Website*

Our scraper will probably assume that every page on the site follows the same HTML structure, but some pages might be laid out differently. It will assume that document links actually point to real files, but some could be broken. It will assume the server responds quickly, but it might be slow or unavailable. It will assume downloaded files like PDFs and Word documents are not corrupted, but they could be. It will also assume URLs stay the same, but the site might use redirects or change its links over time.

Team Practices

9. *Handling a Last-Minute Bug*

If a critical bug was discovered 48 hours before deadline, our first priority will be diagnosis and containment rather than rushing into coding resolution. The team member who discovered the bug would draft a minimal failing test case to reproduce the defect for the rest of the team to see and understand. Then we will have an online call to assess the severity and impact of the bug. If the bug completely breaks the core CLI scraping or querying, we will implement the fix on a dedicated branch. If a full architectural fix carries too much risk so close to the deadline, we will implement a safe temporary workaround where appropriate and document the limitation such as catching this unexpected exception, emit a clean diagnostic message to standard error to keep the stdout clean and ensure the program exits gracefully. Code review is compulsory and conducted by at least one other teammate before merging the bug-fixed branch to main.

10. *Definition of "Finished"*

For a feature to be considered “Finished”, following conditions must be satisfied:

- Fully implement the requirements and run cleanly without errors in the target Ubuntu 22.04 environment
- Include appropriate unit and integration tests covering the required functionality, with all required tests passing
- Keep standard output clean for querying results while routing logs to stderr
- Include appropriate docstrings for all public methods and pass our type-checking checks

11. *Resolving Disagreements*

When team members disagree on whether the software is working as the requirement described, we will have a group discussion on the objective evidence rather than subjective opinions. We will first look back into the project specifications, LMS announcements, official FAQs, and feedback from the unit coordinators on the project specifications clarifications; to check for explicit requirements. If the issue stems from an unstated edge case, each member will write a concrete test case to demonstrate the expected vs the actual behavior. If the specification remains ambiguous, we will decide on the most defensive, user-friendly approach, document it explicitly as a numbered assumption (e.g. A001, A002) in our project documentation as outlined in the project guidelines or post a clarification question on the LMS discussion forum for our facilitator.

Code Quality

12. *Static Analysis and Linting*

We will use **mypy** as our type checker and **ruff** as our linter and formatter. This will be the group's first time using either tool, as it will be with uv (Question 4).

Type checking directly addresses the disadvantage we identified with Python in Question 3. Because Python does not enforce our type annotations at compile time, mistakes such as passing a string where a list is expected, or using a value that a function may return as None, are only discovered when that line happens to execute. This matters for a scraper, because functions that search HTML commonly return None when nothing matches, and that path may never be reached by our normal test data. Type annotations checked by mypy let us find these faults in the source code without running it.

We chose mypy over pyright because it installs as an ordinary Python package through uv alongside our other development dependencies, rather than requiring a separate Node.js installation. We chose ruff because it replaces pylint, black and isort with a single tool, runs quickly, and is configured in the same pyproject.toml file as the rest of our project.

Because neither tool is familiar to us, we are being deliberate about how we introduce them. We will begin with mypy's default settings rather than strict mode, and with ruff's default rule set, then tighten both as we become more comfortable. Enabling every available check from the outset would produce a large volume of findings that obscure the ones that indicate defects, and a tool that produces noise tends to get ignored. One member will take responsibility for the initial configuration and share a short setup note with the group. All configurations are committed to the repository so that every member gets identical results locally and in CI.

13. *Consequences of Failing Checks*

We will treat the checks as blocking rather than advisory, with a defined exception process and an initial grace period.

A pull request cannot be merged into main if the pytest suite, mypy type checking, or ruff linting fails. The author is responsible for fixing their own branch and pushing again; other members are not expected to fix it for them, and no fault is attached to a failed check. Automating this means a tool raises the problem rather than a teammate having to, which keeps the discussion technical.

Because mypy and ruff are new to all of us, the checks will run in **advisory mode for the first week** while members configure their local environments and learn to read the output, becoming blocking from a later date. Making unfamiliar checks blocking before people can run them locally would only create frustration and pressure to work around them.

Where a check produces a false positive, it may be suppressed with an inline comment (for example **# type: ignore[arg-type]**) provided the comment explains why, and the reviewer approves it as part of the review process described in Question 20. Without this exception, a spurious failure close to a deadline would create pressure to disable the checks entirely.

Testing Framework

14. *Test Framework*

We will use **pytest**. Unlike our build tool and static analysis choices, this is familiar ground: all members of the group have used pytest previously.

We expect to rely on parameterised tests (@pytest.mark.parametrize) to run the same test logic across many different inputs, fixtures for shared setup such as temporary directories, pytest.raises to confirm our error handling actually triggers on broken links and bad responses, and markers to separate fast offline tests from slower ones.

pytest alone is not sufficient for the components we identified as hardest to test in Question 7. Tests that make real requests to the policy library would be slow, would fail for reasons unrelated to our code, and would produce different results on different days. We will therefore use responses or requests-mock to supply fake HTTP responses, together with a set of saved HTML pages committed to the repository as test data. This puts both the input to our scraper and its observed output under our control, which is what makes those components testable at all.

CI / Automation

15. *Continuous Integration*

We will set up continuous integration using **GitHub Actions**, configured at the start of the project rather than left until the end. This will be the group's first time using continuous integration.

The workflow will run on every push and on every pull request targeting main. It will check out the repository, install uv, install our dependencies, and then run ruff, mypy and the pytest suite. We will use an Ubuntu 22.04 runner with the Python version specified for the project, so that environment-specific problems are found by us rather than by the marker. After the initial one-week advisory period described in Question 13, branch protection on main will require this workflow to pass, making the policy enforceable rather than only agreed in principle.

Tests that require the live website will be excluded from this workflow and run as a separate scheduled job. Their purpose is different: rather than checking our own code, they detect the site changing underneath us, which we identified in Question 6 as one of the external factors most likely to break the finished software.

Testing Mindset

16. *How Would Someone Break Your Software?*

They would likely try unusual or unexpected inputs and situations that the scraper was not designed for. For example, they could provide an invalid URL, a page with missing information, unexpected HTML structure, duplicate data, empty fields, or a page that cannot be reached. They may also try pages that load slowly or return an error. These cases could reveal weaknesses in our input validation, exception handling, and assumptions about the structure of the website.

17. *Verifying Correct Output*

We will compare the scraper's output against data that we manually verify from known webpages. For selected test pages, we can determine the expected results beforehand and check whether the scraper produces exactly those results. Automated tests will also check individual components where possible. This gives us a known expected result rather than assuming that any successfully generated output is correct.

18. *Evidence of Reliability*

Our confidence would increase if the scraper consistently passes automated tests covering normal cases, edge cases, and failure conditions. We would also test it on several different webpages and compare the results with manually verified data. Successful regression testing after code changes, appropriate handling of invalid or missing data, and successful CI test runs would provide further evidence that the software behaves reliably.

19. *Biggest Challenge*

The biggest challenge is likely to be dealing with the variability and unpredictability of the website being scraped. Our scraper may depend on particular HTML elements, page structures, or data formats, and even small differences between pages could cause failures or incorrect results. Designing the scraper to handle missing, changed, or unexpected data while still producing correct results will therefore be one of the main challenges in ensuring reliability.

Process and Standards

20. *Code Review Process*

For our code review process, changes will be made on separate branches and submitted through a GitHub pull request. At least one team member other than the author will review the changes before they are merged into the main branch. The reviewer will check that the code works as intended, follows the agreed coding standards, does not introduce obvious bugs, and passes relevant tests. Minor changes may receive a shorter review, while larger or more important changes will be reviewed more carefully.